

CareSync

AI-Powered Remote Healthcare Platform

3-Month Development Plan

Development Team

Backend & DevOps: Ramsey

Frontend & UI/UX: Abbie

Timeline: 3 Months (12 Weeks)

Target: Pilot Launch with Healthcare Agencies

Development Plan Generated: June 03, 2026

Project Overview

CareSync is an AI-powered remote healthcare platform focused on elderly and chronically ill patients. The platform enables continuous care from home through health data logging, remote monitoring, video consultations, and AI-generated insights.

Core Features

Feature	Description
Health Data Logging	BP, glucose, weight, temperature, O2 levels, symptoms
Remote Monitoring	Real-time dashboards for doctors and caregivers
Video Consultations	Built-in video calls via Daily.co integration
AI Summaries	Automated consultation summaries and trend analysis
Smart Alerts	Rule-based notifications for critical readings
Appointment System	Scheduling and automated email reminders

Updated Architecture — Agency > Hospital Model

What changed: The original plan had a flat Agency → Users structure. The updated design inserts a Hospital layer between Agency and users, giving each agency the ability to manage multiple hospitals under one account. All user roles (Admin, Doctor, Caregiver, Patient) now belong to a Hospital. Doctors are the one exception — they can be linked to *many* hospitals via a many-to-many join table.

Hierarchy

Role	Responsibilities
Agency	Top-level tenant. Owns billing, quotas, and multiple hospitals.
Hospital	Child of Agency. Parent of all users. One agency → many hospitals.
Admin	Belongs to one Hospital. Manages users, settings, analytics.
Doctor	Linked to many Hospitals (many-to-many). Conducts consultations, reviews data.
Caregiver	Belongs to one Hospital. Assigned to many patients. Logs health data (mobile-first).
Patient	Belongs to one Hospital. Has one primary doctor, many caregivers.

Key relationships

- Agency 1 → * Hospital
- Hospital 1 → * Admin, Caregiver, Patient

- Doctor * ↔ * Hospital (doctor_hospital join table)
- Caregiver * ↔ * Patient (caregiver_patient join table)
- Patient 1 → 1 primary Doctor (foreign key)

Impact on the 12-week timeline: minimal. Week 1 schema gains a hospitals table and two join tables (~half a day). Week 8 Admin Panel adds one hospital-list view. All other weeks, deliverables, and the budget remain unchanged.

Technology Stack

Component	Technology	Why	Owner
Backend	Python FastAPI	Fast, modern, type-safe	Ramsey
Database	PostgreSQL	Structured health data, HIPAA-ready	Ramsey
Frontend	Next.js + React	Modern, fast, SEO-friendly	Abbie
Styling	Tailwind CSS	Rapid UI development	Abbie
Video	Daily.co API	10 k free minutes/month	Ramsey
AI	OpenAI GPT-4o-mini	Cost-effective summaries	Ramsey
Hosting	Railway/Render+Vercel	Free tiers available	Both
Email	Resend / SendGrid	Free tier for alerts	Ramsey

Month 1 — Foundation & Core Features

Weeks 1–4: Authentication, Patient Management, Health Data Logging, Dashboard & Alerts

Week 1 — Setup & Authentication

Backend — Ramsey

- Project setup (FastAPI + PostgreSQL)
- Database schema — Agency, Hospital, User, join tables
- User authentication (JWT)
- Multi-tenant setup with Agency → Hospital isolation
- User roles: Admin, Doctor, Caregiver, Patient
- Basic CRUD endpoints for users
- Docker setup for local development
- Environment configuration

Frontend — Abbie

- Next.js project setup
- Tailwind CSS configuration
- Design system / component library (shadcn/ui)
- Login / Register pages
- Basic routing structure
- API client setup (axios / fetch)
- Authentication state management

Deliverable: Working auth system with role-based access

Week 2 — Patient Management

Backend — Ramsey

- Patient model & database schema
- CRUD endpoints: Create, List, Get, Update, Delete
- Patient-caregiver-doctor relationships
- Pagination & filtering
- Input validation (Pydantic models)

Frontend — Abbie

- Patient list page (dashboard)
- Add new patient form
- Patient detail view
- Edit patient form
- Search & filter UI
- Responsive mobile design (caregivers use phones!)

Deliverable: Complete patient management (CRUD)

Week 3 — Health Data Logging

Backend — Ramsey

- Health readings model (BP, glucose, weight, temp, O2, heart rate)
- logged_by_user_id field — tracks who entered the reading
- Endpoints: Log readings, Get history, Get latest vitals
- Photo / file upload (medication bottles, wounds)
- Validation rules (BP range, glucose range)
- Trend calculation logic

Frontend — Abbie

- Health data entry form (quick & detailed modes)
- Photo capture / upload
- Reading history view
- Trend graphs using Recharts (BP, glucose, weight over time)
- Mobile-optimized input

Deliverable: Health logging with graphs

Week 4 — Dashboard & Alerts

Backend — Ramsey

- Alert system architecture
- Rule-based alerts (high/low BP, glucose, missed readings)
- Alert endpoints (List, Acknowledge)
- Email notification system (SendGrid / Resend)
- Alert prioritization (critical, warning, info)

Frontend — Abbie

- Doctor / Caregiver dashboard with patient list
- Alert feed (critical alerts highlighted)
- Quick stats (total patients, alerts today, consultations)
- Alert notification UI (toast notifications, alert center)
- Patient status cards (healthy, needs attention, critical)
- Dashboard filters (by risk level, last contact)

Deliverable: Complete dashboard with real-time alerts

Month 2 — Video & AI Features

Weeks 5–8: Video Consultations, AI Summaries, Scheduling, Admin Panel

Week 5 — Video Consultations

Backend — Ramsey

- Daily.co API integration
- Consultation scheduling model
- Endpoints: Schedule, List, Start (generate room), Complete
- Store consultation metadata (duration, participants, notes)
- Recording webhook handler (if using Daily.co recording)

Frontend — Abbie

- Schedule consultation form (date/time picker, select patient/doctor)
- Consultation list view (upcoming & past)
- Video call UI (embed Daily.co, in-call vitals display)
- End call button
- Mobile video optimization

Deliverable: Complete video consultation flow

Week 6 — AI Summaries (your differentiator!)

Backend — Ramsey

- OpenAI API integration (GPT-4o-mini)
- Consultation summary: doctor notes + vitals → summary
- Weekly health summary: 7 days of readings → trend analysis
- Endpoints: Generate consultation summary, Weekly report
- Prompt engineering for medical context
- Error handling & retry logic
- Summary storage in database

Frontend — Abbie

- Consultation summary view (display AI summary after call)
- Edit / annotate summary, save final version
- Weekly report view (auto-generated, trend highlights)
- AI loading states
- Summary history

Deliverable: AI summary generation working

Week 7 — Scheduling & Reminders

Backend — Ramsey

- Appointment scheduling system
- Doctor availability management
- Endpoints: Create, List, Reschedule, Cancel appointments
- Automated reminder system (cron: 24 hrs & 1 hr before)
- Email reminders (no SMS yet — too expensive)
- Daily health check-in reminders

Frontend — Abbie

- Appointment booking flow (calendar view, time-slot selection)
- Appointment management (reschedule, cancel)
- Reminder preferences settings
- Calendar integration (export to Google Calendar)

Deliverable: Scheduling with automated reminders

Week 8 — Admin Panel & Agency Management

Backend — Ramsey

- Agency (tenant) management endpoints
- Hospital management endpoints (create, list hospitals per agency)
- Endpoints: Get stats, List patients/users per hospital
- Billing / usage tracking (patients, consultations, AI summaries)
- Analytics endpoints (patient growth, consultation frequency, alerts)

Frontend — Abbie

- Admin dashboard (agency overview, hospital list, user management)
- Hospital selector — switch between hospitals in the UI
- Analytics views (charts for growth, consultations, alerts)
- Usage metrics (for billing discussions)
- Settings panel

Deliverable: Admin panel for agencies

Month 3 — Polish, Testing & Pilot Launch

Weeks 9–12: Optimization, Testing, Deployment, Launch

Week 9 — Mobile Optimization & UX Polish

Backend — Ramsey

- API performance optimization (indexing, query optimization, caching)
- Rate limiting
- API documentation (FastAPI auto-docs + examples)
- Error handling improvements
- Logging & monitoring setup

Frontend — Abbie

- Mobile responsiveness audit (test on real devices)
- Progressive Web App (PWA) setup (offline, add to home screen)
- UX improvements (loading states, error messages, tooltips)
- Accessibility audit (basic)

Deliverable: Mobile-optimized, polished UI

Week 10 — Testing & Bug Fixes

Backend — Ramsey

- Unit tests (auth, alerts, AI summaries)
- Integration tests (API endpoints, database)
- Security audit (SQL injection, XSS, CORS, encryption)
- Load testing (basic)
- Bug fixes from frontend testing

Frontend — Abbie

- Manual testing across all flows
- Cross-browser testing
- Mobile device testing
- Create bug list & fix critical bugs
- User acceptance testing (UAT with friends/family)

Deliverable: Tested, bug-free system

Together (Week 10):

- End-to-end testing across all user roles
- Test scenarios: patient logs reading → alert fires → doctor acknowledges
- Integration bug fixes

Week 11 — Deployment & Documentation

Backend — Ramsey

- Production database setup (PostgreSQL on Railway/Render/Supabase)
- Environment variables management
- Backend deployment (Railway / Render)
- SSL / HTTPS setup
- Database backups setup
- Monitoring (Sentry free tier)
- CI/CD pipeline (GitHub Actions — basic)
- API documentation finalization

Frontend — Abbie

- Frontend deployment (Vercel)
- Environment configuration (API URLs)
- Custom domain setup (if available)
- Performance optimization (images, code splitting, lazy loading)
- Analytics setup (PostHog / Plausible free tier)
- User documentation / help section

Deliverable: Deployed system on custom domain

Together (Week 11):

- Full system smoke test
- Demo account with sample data (agency → hospital → patients)
- Pitch deck / demo video

Week 12 — Pilot Launch & Iteration

Backend — Ramsey

- Onboard first pilot agency and its hospitals
- Set up agency account, hospitals, and users
- Train administrators
- Monitor system performance
- Quick bug fixes
- Collect feedback
- Plan iteration 2

Frontend — Abbie

- Create onboarding materials (videos, guides, FAQ)
- User support (respond to questions)
- Track user behavior (most-used features)
- UI tweaks based on feedback
- Iterate on confusing flows

Deliverable: Live pilot with real users!

Together (Week 12):

- Weekly pilot review meetings
- Document learnings
- Prioritize iteration 2

Success Metrics

Month	Goals
Month 1	Auth & patient management complete Health data logging works end-to-end
Month 2	Video calls working AI summaries generating Alert system functional
Month 3	Live pilot with 1 agency / hospital 20–50 patients in system Feedback collected for v2

Tools & Workflow

Project Management	GitHub Projects or Trello (free) — task tracking
Version Control	GitHub — code repository
Communication	Discord / Slack — daily async standups
Design	Figma (free) — UI mockups
Documentation	Notion / Google Docs
API Testing	Postman / Insomnia
Development	VS Code, Docker for local environment

Daily Routine

- Morning — 15-min async standup (Discord/Slack text update)
- What you did yesterday / doing today / any blockers
- Evening — push code, update task board
- Weekly — 1-hour sync call (progress review, plan next week)

Budget Breakdown (Monthly)

Service	Free Tier	MVP Cost	When Scaling
Daily.co (video)	10 k min/month	\$0	\$24/month (100 patients)
OpenAI (AI)	Pay-as-you-go	~\$3–6/mo	Scales with usage
Railway/Render (backend)	Free tier	\$0	\$5–10/month
Vercel (frontend)	Free hobby tier	\$0	Free
Supabase (database)	500 MB free	\$0	\$25/month (growth)
SendGrid (email)	100/day free	\$0	Free for MVP

Total MVP Cost	\$0–10/month		~\$60–75/month (growth)
----------------	--------------	--	-------------------------

Risk Mitigation Strategies

Situation	Mitigation Strategy
Weeks 1–4 behind	Skip admin panel (Week 8), focus on core patient features
Weeks 5–7 behind	Simplify AI summaries (use templates instead of full AI)
Weeks 9–12 behind	Launch with known bugs, fix iteratively with pilot feedback
Frontend struggles	Pair programming on complex components, use shadcn/ui templates
Money runs out	Daily.co free tier → WebRTC; reduce AI summary frequency; use all free tiers

Pro Tips for Success

- Start with the hardest part first — Video + AI in Month 2 prevents last-minute crunch.
- Deploy early — don't wait till Week 11; deploy by Week 4 to catch issues early.
- Get real users by Week 8 — even 5 patients from family/friends for testing.
- Document as you go — future you will thank present you.
- Don't over-engineer — ship features quickly, iterate based on feedback.
- Communicate daily — 15-min async updates prevent blockers.
- Celebrate small wins — deployed auth? Celebrate! First video call? Celebrate!
- Ask for help — stuck for >2 hours? Reach out to communities (Discord, Reddit).

Next Steps

1. Review this updated plan together and confirm the Agency → Hospital hierarchy.
2. Set up communication channels (Discord / Slack).
3. Create GitHub repository and project board.
4. Design database schema on Day 1 of Week 1 — use the models.py file as the starting point.
5. Start coding!